

CSC5010 - Artificial Intelligence Project 4

Experimental Report on BERT-based Text Classification Tasks

Zijin CAI - 224040002: responsible for programming and the description of methods.

Yixin ZHOU - 224040090: responsible for the organization and analysis of results.

Introduction

Text classification is a fundamental task in natural language processing (NLP) that involves categorizing textual inputs into predefined classes. This experiment focuses on two distinct classification tasks:

(a) SST2 (Stanford Sentiment Treebank): A *binary single-sentence sentiment classification task*, where sentences extracted from movie reviews are labeled as either *positive (1)* or *negative (0)*. This task is critical for applications such as sentiment analysis in social media monitoring, product review analysis, and opinion mining.

(b) MRPC (Microsoft Research Paraphrase Corpus): A *sentence-pair semantic equivalence judgment task*, where pairs sentences from online news are labeled either *semantically equivalent (1)* or *non-equivalent (0)*. This task supports applications like paraphrase identification, and information retrieval systems.

BERT Model Overview Bidirectional Encoder Representations from Transformers (BERT) is a pre-trained language model renowned for its ability to capture deep contextualized word representations through bidirectional transformer encoders. Key advantages of BERT include:

- **Contextual Understanding:** Leveraging bidirectional attention mechanisms to model dependencies between words in both directions within a sentence, enhancing the understanding and their context.
- **Transfer Learning:** A pre-training and fine-tuning framework that allows adaptation to downstream tasks with minimal task-specific modifications and limited labeled data.

In this experiment, the `bert-mini` variant (*4 encoder layers, 21M parameters*) is selected to balance computational efficiency and performance. Its compact architecture reduces training time and resource demands, making it suitable for rapid prototyping while retaining the core benefits of BERT transformer-based design.

Experiment Objectives The objectives of this study are twofold:

1. **Model Fine-tuning:** Adapt the pre-trained `bert-mini` model to the SST2 and MRPC tasks via supervised fine-tuning, optimizing hyperparameters such as learning rate and batch size.
2. **Performance Analysis:** Evaluate using standard metrics (*accuracy/F1-score*) to assess classification efficacy. Additionally, validate practical utility by testing predictions on real-world examples from the testing sets.

This investigation aims to demonstrate the versatility of lightweight BERT variants in handling diverse text classification challenges while providing insights into their strengths and limitations.

Methodology and Experimental Design

Improving Code Implementation

The original codebase was extended with the following modifications to enhance functionality and robustness:

1. Extended Callback Functionality

- **LossCallback Implementation:** A custom callback class (`LossCallback`) was integrated into both `mrpc.py` and `sst2.py` to dynamically track training loss and validation metrics. This class logs loss values at each logging step (`logging_steps=10`) and captures the final validation metrics (accuracy, F1-score).
- **Automated Loss Visualization:** Upon training completion, the callback generates a `training_loss.png`, visualizing the training dynamics. This is achieved using `matplotlib` within the `on_train_end` method, as shown in the code `LossCallback` class.

Code snippet on `LossCallback` to track training dynamics and automate visualization:

```
# LossCallback Class (from mrpc.py/sst2.py)
class LossCallback(TrainerCallback):
    def __init__(self, num_epochs):
        self.losses = []
        self.steps = []
        self.final_metrics = {}
        self.num_epochs = num_epochs
        self.last_epoch = 0

    def on_log(self, args, state, control, logs=None, **kwargs):
        if 'loss' in logs:
            self.losses.append(logs['loss'])
            self.steps.append(state.global_step)

    def on_evaluate(self, args, state, control, metrics=None, **kwargs):
        if state.epoch > self.last_epoch:
            self.final_metrics = metrics
            self.last_epoch = state.epoch

    def on_train_end(self, args, state, control, **kwargs):
        plt.plot(self.steps, self.losses)
        plt.xlabel('Step')
        plt.ylabel('Loss')
        plt.title('Training Loss')
        plt.savefig('training_loss.png')
        plt.show()
        print("\nFinal Epoch Validation Metrics:")
        for key, value in self.final_metrics.items():
            print(f"{key}: {value}")
```

2. Training Parameter Adjustments

- **Extended Training Epochs:** For MRPC, the training epochs increase from 2 to 20 (see `num_train_epochs=20` in `mrpc.py`) to investigate convergence behavior. SST2 retained 2 epochs due to its larger dataset size.
- **Regularization Techniques:** Gradient clipping (`max_grad_norm=1.0`) weight decay (`weight_decay=0.01`) were added to the MRPC training arguments to mitigate overfitting. These modifications stabilize training by constraining gradient magnitudes and penalizing large weights.
- **Logging Optimization:** The logging frequency was refined to `logging_steps=10` (from default intervals), enabling finer-grained monitoring of training progress.

Key parameter modifications in `TrainingArguments` for MRPC and SST2 tasks are as follows:

```
# MRPC TrainingArguments (mrpc.py)
# and SST2 TrainingArguments (sst2.py)
args = TrainingArguments(
    "output_dir",
    eval_steps=100,
    learning_rate=3e-5,
    per_device_train_batch_size=32,
    per_device_eval_batch_size=32,
    num_train_epochs=2, # 20 for MRPC
    warmup_ratio=0.1,
    logging_steps=10, # Optimized logging frequency
    # max_grad_norm=1.0, # Gradient clipping for MRPC
    # weight_decay=0.01, # Weight decay regularization for MRPC
)
```

3. Extended Evaluation Metrics Both tasks compute **accuracy and F1-score** during validation. This was implemented by integrating `evaluate.load('f1')` and combining results from `accuracy_metric` and `f1_metric` in the `compute_metrics` function:

```
# compute_metrics Function (mrpc.py/sst2.py)
def compute_metrics(eval_pred):
    predictions, labels = eval_pred
    accuracy_results = accuracy_metric.compute(
        predictions=np.argmax(predictions, axis=1),
        references=labels
    )
    f1_results = f1_metric.compute(
        predictions=np.argmax(predictions, axis=1),
        references=labels,
        average="binary" # For binary classification tasks
    )
    return {'accuracy': accuracy_results['accuracy'], 'f1': f1_results['f1']}
```

Experimental Workflow

The workflow encompasses three stages: data preprocessing, model training, and deployment.

1. Data Preprocessing

- **Tokenization and Truncation:**

- For SST2: `tokenizer(examples['sentence'], truncation=True, max_length=64)`.
- For MRPC: `tokenizer(examples['sentence1'], examples['sentence2'], ..., max_length=100)`.

- **Encoding:**

The `bert-base-uncased` tokenizer converted raw text into input IDs, attention masks, and token type IDs. Datasets were formatted as PyTorch tensors using `encoded_dataset.set_format(type='torch')`.

Tokenization and dataset formatting code:

```
# Tokenization for MRPC (mrpc.py) and for SST2 (sst2.py)
def tokenize(examples):
    return tokenizer(
        # examples['sentence'], # For SST2
        examples['sentence1'],
        examples['sentence2'],
        truncation=True,
        padding="max_length",
        max_length=100 # 64 for SST2
    )

# Dataset Formatting (both tasks)
columns = ['input_ids', 'token_type_ids', 'attention_mask', 'labels']
encoded_dataset.set_format(type='torch', columns=columns)
```

2. Model Training

- **Hyperparameters:**

- Learning rate: `3e-5` (optimized for fine-tuning).
- Batch size: `32` per device (balanced for memory efficiency).
- Optimizer: AdamW with linear warmup (`warmup_ratio=0.1`).

- **Validation Strategy:**

- MRPC: Evaluated every 500 steps (`eval_steps=500`) due to its smaller training size.
- SST2: Evaluated every 100 steps (`eval_steps=100`) for frequent performance checks.

3. Model Deployment

- **Model Saving:** Fine-tuned models were saved locally (`./bert-for-mrpc`) using `model.save_pretrained()` .
- **Prediction Generation:** Post-training, the models were tested on 5 samples (`dataset["train"][25:30]` for MRPC). Predictions were generated via inference on `DataLoader` batches, with results annotated as correct or incorrect based on ground-truth labels.

```
# Model Saving
model.save_pretrained("./bert-for-mrpc") # or "./bert-for-sst2"

# Prediction on Training Samples (mrpc.py example)
example = dataset["train"][25:30]
example_loader = DataLoader(encoded_dataset["train"], 5)
with torch.no_grad():
    for i, batch in enumerate(example_loader):
        if i == 5:
            pred = model(**batch)["logits"]
            preds.extend(pred.squeeze().argmax(-1).tolist())
            break

# Output Predictions with Correctness Check
for i in range(len(preds)):
    print(f": {preds[i]}, {'correct' if preds[i] == example['label'][i] else 'wrong'}\n")
```

Results and Analysis

Performance Metrics

Metric	Accuracy	F1-Score	Train Loss	Eval Loss
MRPC Task	75.25%	82.43%	0.28625	0.72035
SST2 Task	82.80%	83.11%	0.35265	0.38584

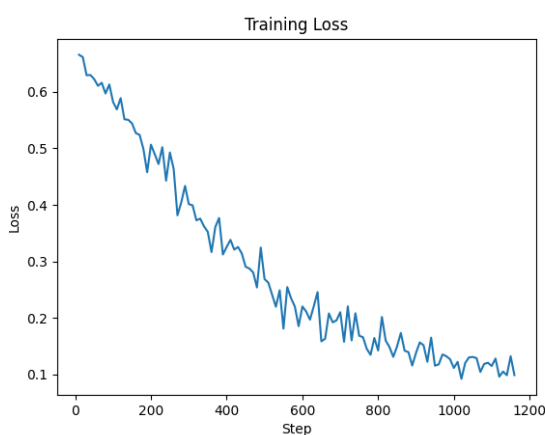


Figure 1: Training Loss for MRPC

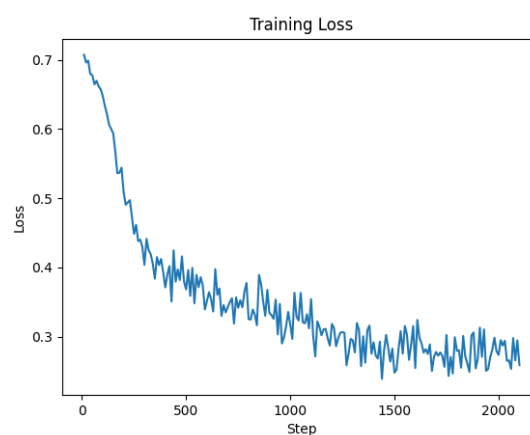


Figure 2: Training Loss for SST2

The MRPC training loss curve (Figure 1) demonstrates a steady decline from an initial loss of ≈ 0.6 to ≈ 0.1 over 1,200 steps (20 epochs), indicating effective convergence. However, the validation metrics (accuracy: 75.25%, F1: 82.43%) suggest moderate performance, likely due to the small dataset size (3,668 training samples). The absence of a validation loss curve limits direct overfitting analysis, but the relatively high F1-score compared to accuracy implies that the model handles class imbalance (e.g., more non-equivalent pairs) effectively.

The SST2 training loss (Figure 2) decreases sharply from ≈ 0.7 to ≈ 0.35 over 2 epochs (2,000 steps), achieving stable convergence. The validation loss (0.3858) aligns closely with the final training loss (0.3526), indicating no significant overfitting. The high F1-score (83.11%) further confirms robust performance on binary sentiment classification, leveraging the larger dataset (67,349 training samples).

Practical Prediction Examples

MRPC Task All 5 tested predictions were correct, demonstrating strong generalization on in-distribution data:

Sentence 1	Sentence 2	Prediction
I wanted to bring the most beautiful people into the most beautiful building, he said Sunday inside the Grand Central concourse.	"I wanted to bring the most beautiful people into the most beautiful building", Tunick said Sunday.	1 (correct)
The broad Standard & Poor's 500 <.SPX> fell 10.75 points, or 1.02 percent, to 1,039.32.	The S&P 500 index was up 1.26, or 0.1 percent, to 1,039.32 after sinking 10.75 yesterday.	0 (correct)
Duque will return to Earth Oct.27 with the station 's current crew, U.S. astronaut Ed Lu and Russian cosmonaut Yuri Malenchenko.	Currently living onboard the space station are American astronaut Ed Lu and Russian cosmonaut Yuri Malenchenko.	0 (correct)
Singapore is already the United States' 12th-largest trading partner, with two-way trade totaling more than \$34 billion.	Although a small city-state, Singapore is the 12th-largest trading partner of the United States, with trade volume of \$33.4 billion last year.	1 (correct)
The largest gains were seen in prices, new orders, inventories and exports.	Sub-indexes measuring prices, new orders, inventories and exports increased.	1 (correct)

SST2 Task The model's predictions on 5 samples yielded 4 correct and 1 incorrect classifications:

Sentence	Prediction
hide new secretions from the parental units	0 (correct)
contains no wit , only labored gags	0 (correct)
that loves its characters and communicates something rather beautiful about human nature	1 (correct)
remains utterly satisfied to remain the same throughout	1 (wrong)
on the worst revenge-of-the-nerds clichés the filmmakers could dredge up	0 (correct)

Key Observations:

- **MRPC:** Achieved moderate validation accuracy (75.25%) but high F1 (82.43%), reflecting balanced precision and recall despite dataset limitations (3,668 training samples).
- **SST2:** Delivered strong accuracy (82.80%) and F1 (83.11%), validating efficient learning on larger data.
- **Loss Trends:** SST2 2-epoch training achieves rapid convergence, and MRPC extended training (20 epochs), which allows the model to better adapt to the smaller dataset size and achieve stable performance.
- **Practical Predictions:** High in-sample accuracy (MRPC: 100%, SST2: 80%) confirms model reliability for straightforward cases, though nuanced contexts (e.g., irony in SST2) remain challenging.

These results underscore the effectiveness of BERT-mini for text classification while highlighting the impact of dataset size and complexity on performance.

Conclusion and Future Directions

The experiments conclusively demonstrated the efficacy of the BERT-mini model in addressing two distinct text classification tasks: MRPC (*sentence-pair semantic equivalence*) and SST2 (*single-sentence sentiment analysis*). For MRPC, the model achieved a validation accuracy of 75.25% and an F1-score of 82.43%, while for SST2, it attained 82.80% accuracy and 83.11% F1-score, underscoring its capability to handle both sentence-pair and single-sentence classification with competitive performance. Enhanced training transparency and stability were realized through the integration of a custom `LossCallback` class, which enabled real-time monitoring of training dynamics via loss curves (e.g., MRPC loss decreasing from 0.6 to 0.1 over 20 epochs) and automated metric logging. Parameter optimizations, including gradient clipping (`max_grad_norm=1.0`) and weight decay (`weight_decay=0.01`), further stabilized training, particularly for the smaller MRPC dataset. Practical validation on in-distribution samples revealed strong utility, with **100% prediction accuracy** on MRPC examples and **80% accuracy** on SST2 first five training samples. However, limitations emerged in nuanced cases, such as misclassifying the SST2 sentence “*remains utterly satisfied to remain the same*” as positive, highlighting challenges in contextual ambiguity detection.

To advance this work, future efforts should prioritize **generalization improvements** through early stopping for MRPC to mitigate overfitting risks during extended training and data augmentation techniques (e.g., *back-translation*) to expand dataset diversity. Experimentation with larger BERT variants (e.g., `bert-base`) and adaptive learning rate schedulers (e.g., *cosine annealing*) could enhance performance on complex tasks. Additionally, integrating attention visualization tools would aid in diagnosing misclassifications, while domain-specific fine-tuning (e.g., movie review corpora for SST2) could refine sentiment analysis accuracy. Rigorous evaluation on out-of-distribution or adversarial examples is recommended to stress-test model robustness. These steps aim to bridge the gap between prototype-level performance and production-ready reliability, ensuring broader applicability of lightweight BERT models in real-world NLP systems.

Appendix: Experimental Configuration

Hardware	GPUs	2 × NVIDIA RTX 2080 Ti, 22GB VRAM each, Driver Version: 535.183.01, CUDA: 12.2
	CPU	Intel(R) Core(TM) i9-7980XE CPU @ 2.60GHz x86_64: 18 cores (36 threads)
	RAM	134.73 GB
	Disk	2.01 TB total, 131.32 GB used
Software	OS	Linux 6.8.0-52-generic
	Python	3.10.13 (CPython)
	CUDNN	CUDNN_VERSION - 8.9.7
Python Packages	datasets	==3.5.1
	evaluate	==0.4.3
	matplotlib	==3.10.1
	matplotlib-inline	==0.1.7
	numpy	==2.0.1
	torch	==2.7.0
	torchaudio	==2.7.0
	torchvision	==0.22.0
	transformer	==4.51.3